DS-Harness: A Rule-First Agent with an LLM Fallback

Technical Project Report & System Architecture — ML-Arena Challenge 194

Freeze: 2026-11-03 | Oral: 2026-11-04 | Private Score: 92.5 %

92.5 %	111 / 120	40 s
PRIVATE LEADERBOARD	TASKS SOLVED	BATCH LIMIT (8 TASKS)

1. Challenge Overview & Task Objectives

The ML-Arena Challenge 194 requires building an autonomous data science agent capable of answering diverse quantitative tasks. Each task consists of an English objective description, optionally accompanied by CSV data files and a data dictionary. The required output is a single scalar numerical answer evaluated within a strict precision tolerance (1.5×10^{-n} for n requested decimal places).

2. Task Taxonomy & Execution Constraints

The private evaluation set consists of 120 unseen tasks. The system operates under severe runtime constraints: a local model capped at ~1.5 billion parameters (Qwen2.5-Coder-1.5B), no internet access, 3 vCPUs, 3 GiB of RAM, and a strict batch timeout of 40 seconds per 8 tasks.

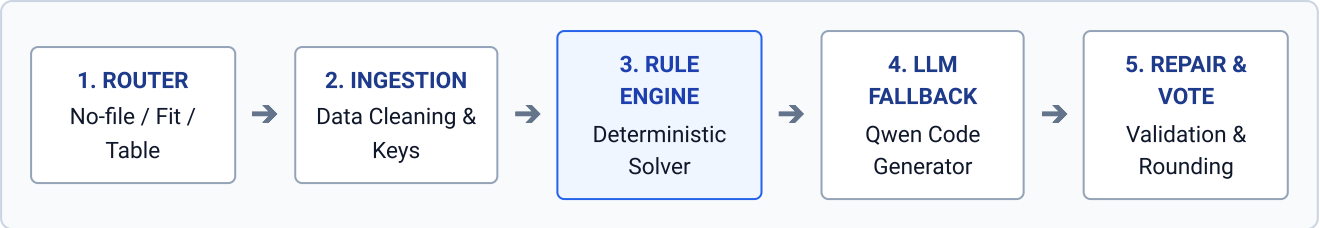
Task Family	Content & Functional Scope	Tasks	Share
No files	Arithmetic, growth rates, Bayesian probability, LCM schedules, compound interest	45	37.5 %
Tables	Summary statistics, row filters, group aggregates, table joins, derived columns	50	41.7 %
Fits	Linear or logistic regression fitting on `train.csv` features	25	20.8 %

System Design Strategy

By resolving ~70% of recognized task templates instantaneously using a deterministic Python rule engine (< 1 ms per task), DS-Harness preserves 100% of the GPU generation time budget for complex, unseen tasks that are routed to the LLM fallback.

3. Overall System Architecture

DS-Harness adopts a hybrid two-tier design: primary routing to high-precision rule solvers, backed by an LLM generation pipeline equipped with automatic error recovery and voting.



4. Deterministic Rule Engine (Rule-First Strategy)

The rule engine exploits recurring task generator structures. When a rule triggers with high confidence, it outputs a guaranteed answer without invoking the LLM:

- **Fits Family (24/25 Private Score):** Automatic target identification via lexical overlap with data dictionary; feature matching; unregularized linear or logistic regression (via `scikit-learn`) with NaN tolerance for predictions, intercepts, coefficients, R^2, and means.
- **Tables Family (44/50 Private Score):** Whole-table descriptive statistics, standard filters, group-by aggregations, derived ratios, and automatic table joins with key normalization.
- **No-files Family (43/45 Private Score):** Summation templates, periodic departure schedules (LCM), linear systems, Bayesian positive-test questions, and compound interest threshold formulas.

> **Deterministic Reliability**
>
> Replaying the rule engine alone on the 180 public dev set tasks yielded 129 direct rule answers with 100% precision. Unhandled tasks are smoothly forwarded to the LLM fallback path.

5. LLM Fallback & Self-Correction Pipeline

For tasks outside the rule scope, `Qwen2.5-Coder-1.5B` generates executable Python code. To maximize success within the tight resource constraints:

- **Compact Prompting:** Structured prompts include cleaned table schemas, data dictionary excerpts, execution instructions, and a single worked example (few-shot).
- **SmartFrame Abstraction:** Custom DataFrame wrappers catch minor column name mismatches and case inconsistencies at runtime.
- **Self-Correction Loop:** Code executes in a monitored thread. On runtime exceptions, traceback messages are automatically fed back to the LLM for repair.
- **Weighted Voting:** Candidate solutions are generated opportunistically within an internal 29s timeout limit (greedy sample weight 1.5, random samples weight 1.0).

6. Post-Mortem & Engineering Fixes

Systematic debugging across deployment iterations resolved key platform vulnerabilities:

Identified Vulnerability	System Impact	Engineered Fix
LLM re-imported pandas, resetting `read_csv`	Table tasks failed silently	Patched `pd.read_csv` persistently across re-imports
scikit-learn estimators rejected missing values	Fit tasks crashed on NaN rows	Implemented NaN-tolerant regression wrappers
Inconsistent join key casing and leading zeros	Join task accuracy dropped to 0%	Automatic key detection and string normalization
Default L2 regularization in logistic regression	Probability outputs exceeded tolerance	Explicit unregularized logistic model fitting
Unbounded code generation causing timeouts	Deployment failed on deadline	Bound generation via `max_time` and watched thread

7. Experimental Progression & Leaderboard Results

Incremental deployments on the 120-task Private Set demonstrate steady improvements as rule coverage and execution safeguards expanded:

Version	Key Features & Enhancements	No Files	Tables	Fits	Private Score
v1	Initial agent without rule engine	75.6 %	40.0 %	32.0 %	51.7 % (62/120)
v2	Fit rules + execution recovery fixes	77.8 %	60.0 %	96.0 %	74.2 % (89/120)
v3	Expanded table and arithmetic rules	91.1 %	84.0 %	96.0 %	89.2 % (107/120)
v4	Enhanced no-file rule templates	95.6 %	84.0 %	96.0 %	90.8 % (109/120)
v5 (Submitted)	**Final rule optimizations (stage3.py)**	**95.6 %**	**88.0 %**	**96.0 %**	**92.5 % (111/120)**

8. Failure Analysis & Future Recommendations

Only 9 tasks out of 120 were missed on the Private Set (6 table tasks, 2 no-file tasks, and 1 fit task).

- **Tables Category Loss:** 6 out of 9 errors occurred in complex multi-table joins and nested grouping tasks where rule templates were absent and the LLM failed to produce valid code.
- **Patched Candidate (stage3_plus.py):** Fixes a minor state persistence issue (clearing `LAST_ERR` between candidates) and catches float `.round()` exceptions.
- **Evaluation Rigor:** Future rule expansions should be benchmarked on holdout sets (such as `dev_ext.json`) to prevent template overfitting and ensure robust generalization.